	Sanjivani Rural Education Society's **SANJIVANI COLLEGE OF ENGINEERING** **(An Autonomous Institution)** Kopargaon – 423 603, Maharashtra.	**ACAD-F-15 K**
Academic Year: 2026-27	**CIA ACTIVITY REPORT**	**Revision : 00**
Department :	**Computer Engineering**	**Date of Preparation**
Course Code & Name:	**PCCO302 : Computer Network**	**Year/Sem : TY/ 5th**

Activity Title: Adaptive Error Correction for High-Noise Wireless Communication

Submitted by:

Group Id- 11

Roll No	PRN No	Name of the Student	Signature
66	0124UCSM1066	Deokar Sai Sandeep	
68	0124UCSM1068	Palande Siddharth	
166	0125UCSM2012	Deokar Aditya Sandip	
169	CS23M1023	Chavan Aniket Prakesh	

Dr. P. N. Kalavadekar

Course Incharge

ABSTRACT

Wireless communication systems are affected by noise, interference, signal degradation, and channel disturbances, which can cause errors in transmitted data. If corrupted data is not detected or corrected, communication reliability decreases and the quality of the transmitted information is affected. Error-control coding techniques are therefore used in communication systems to introduce controlled redundancy so that the receiver can recover information affected by transmission errors.

The objective of this project is to develop a simulation-based **Adaptive Error Correction System for High-Noise Wireless Communication**. Instead of applying a single fixed error-correction technique for all channel conditions, the proposed system dynamically selects the error-correction strength according to the Signal-to-Noise Ratio (SNR). Three coding levels are implemented. Level 1 uses no error correction for good channel conditions, Level 2 uses Hamming (7,4) error correction for moderate channel conditions, and Level 3 uses Hamming (7,4) combined with triple repetition for highly noisy conditions.

The system generates random binary data, applies the selected coding technique, passes the encoded data through a simulated noisy channel, decodes the received data, and calculates performance parameters such as Bit Error Rate (BER), throughput, redundancy, and number of errors. The adaptive system is also compared with a fixed Hamming (7,4) system.

The simulation uses SNR values from 0 dB to 12 dB. The results demonstrate that the proposed adaptive system changes its coding strength according to channel conditions. At low SNR, stronger protection is selected, while at high SNR the system reduces redundancy and improves transmission efficiency. At 12 dB, the adaptive system achieves a throughput of 96.74% with a redundancy factor of 1.00, compared with 56.73% throughput and 1.75 redundancy for the fixed Hamming system.

The project demonstrates that adaptive error-control selection can provide a useful balance between reliability and transmission efficiency in varying wireless channel conditions.

INDEX

Chapter	Title	Page.no
Chapter 1	Introduction	4
Chapter 2	Literature Survey	7
Chapter 3	System Design	11
Chapter 4	Implementation	20
Chapter 5	Results and Analysis	25
Chapter 6	Conclusion	31
Chapter 7	References	34
Annexure 1	GitHub Upload Screenshot	35
Annexure 2	Presentation Geo-tagged Photo	36

CHAPTER 1 - INTRODUCTION

1.1 Background

Wireless communication has become an important part of modern computer and communication networks. Data is transmitted between devices through wireless channels in applications such as mobile communication, Wi-Fi, satellite communication, IoT systems, sensor networks, and other distributed systems.

During wireless transmission, the transmitted signal may be affected by noise, interference, attenuation, fading, and other channel disturbances. These effects can change transmitted bits from 0 to 1 or from 1 to 0. Such errors may result in corrupted packets and unreliable communication.

Error-control coding is a technique used to improve communication reliability by adding controlled redundancy to the original information before transmission. The receiver uses this additional information to detect or correct errors. Different coding techniques provide different levels of protection and require different amounts of redundancy.

Modern communication systems use advanced coding schemes such as LDPC and Polar codes. For example, 5G New Radio uses LDPC coding for important data channels and Polar coding for selected control information. 3GPP specifies channel coding and related processing in TS 38.212.

However, using strong error correction continuously is not always efficient. When the channel is relatively good, excessive redundancy increases the number of transmitted bits and reduces transmission efficiency. On the other hand, weak error correction under poor channel conditions may result in a high number of errors.

Therefore, this project focuses on an adaptive approach in which the error-correction strength is selected according to the channel condition represented by SNR.

1.2 Problem Statement

In a wireless communication system, channel conditions are not always constant. A communication channel may experience low noise at one time and high noise at another time.

A fixed error-correction technique applies the same coding strength regardless of the channel condition. This creates two major problems:

1. Strong coding in good channel conditions introduces unnecessary redundancy.
2. Weak coding in highly noisy conditions may not provide sufficient protection.

The problem addressed by this project is therefore:

To develop a simulation-based wireless communication system that dynamically selects an appropriate error-correction strength according to channel conditions, thereby balancing data reliability and transmission efficiency.

1.3 Motivation

The main motivation of this project is to improve the efficiency of error-controlled wireless communication.

Traditional systems generally use predefined coding schemes. Although these schemes provide reliable communication, a fixed coding level may not be optimal for every channel condition.

For example, if the channel has a high SNR, the probability of errors is relatively low. In such a condition, using a large amount of redundancy is unnecessary. If the channel has a very low SNR, stronger error protection becomes useful.

The proposed project therefore introduces an adaptive decision mechanism that selects:

- No error correction for good conditions.
- Hamming error correction for moderate conditions.
- Hamming plus repetition for highly noisy conditions.

This approach demonstrates the basic principle of adaptive error-control coding.

1.4 Objectives

The main objectives of the project are:

1. To study the effect of noise on wireless binary data transmission.
2. To implement a simulation of a noisy wireless communication channel.
3. To implement Hamming (7,4) error correction.
4. To implement a stronger protection mechanism using triple repetition.
5. To design an adaptive controller based on SNR.
6. To dynamically select the error-correction level according to channel conditions.
7. To calculate Bit Error Rate (BER).
8. To calculate effective throughput.
9. To measure coding redundancy.
10. To compare adaptive error correction with fixed Hamming (7,4) coding.
11. To analyze the reliability-efficiency trade-off of the proposed approach.

1.5 Scope of the Project

The project is implemented as a software-based communication simulation.

The scope includes:

- Random binary data generation.
- Hamming (7,4) encoding and decoding.
- Triple repetition coding and majority decoding.
- Simplified noisy-channel simulation.
- SNR-based adaptive coding selection.
- BER calculation.
- Throughput calculation.
- Redundancy calculation.
- Comparison between adaptive and fixed coding.
- CSV generation for result analysis.

The current project does not implement an actual RF transmitter or receiver. It is a simulation intended to demonstrate the adaptive error-correction concept.

1.6 Proposed Solution

The proposed system consists of a transmitter, adaptive controller, simulated noisy channel, receiver, and performance-analysis module.

The transmitter generates binary information. The adaptive controller selects one of three coding levels based on SNR.

The three levels are:

Table 1.6.1: Coding levels based on SNR

Level	Coding Method	Condition
Level 1	No Error Correction	SNR ≥ 8 dB
Level 2	Hamming (7,4)	4 dB ≤ SNR < 8 dB
Level 3	Hamming (7,4) + Triple Repetition	SNR < 4 dB

The encoded data is passed through a simulated noisy channel. At the receiver, the appropriate decoder is applied. Finally, BER, throughput, redundancy, and errors are calculated.

CHAPTER 2 - LITERATURE SURVEY

2.1 Introduction

Error-control coding is an important area of digital communication and information theory. The purpose of error-control coding is to improve the reliability of information transmission over an imperfect communication channel.

The fundamental concept of communication over noisy channels was established by information theory, where the capacity of a communication channel depends on channel characteristics such as noise and signal power.

Over time, different error-control techniques have been developed, including parity checking, Hamming codes, convolutional codes, Turbo codes, LDPC codes, and Polar codes.

2.2 Error Control in Wireless Communication

Wireless channels are susceptible to errors caused by noise, interference, fading, attenuation, and other environmental effects.

Two broad approaches are commonly used for error control:

Forward Error Correction

Forward Error Correction (FEC) adds redundancy to the transmitted information. The receiver uses this redundancy to detect and correct errors without requesting retransmission.

Advantages include:

- Reduced need for retransmission.
- Improved reliability.
- Suitable for wireless and high-latency communication.
- Can improve communication performance under noisy conditions.

The disadvantage is that additional redundant bits increase transmission overhead.

Automatic Repeat Request

Automatic Repeat Request (ARQ) detects errors and requests retransmission of corrupted data.

Advantages include:

- High reliability.
- Redundancy is introduced only when retransmission is required.

Disadvantages include:

- Additional delay.
- Increased traffic.
- Retransmission may not be suitable for highly delayed or unreliable links.

The proposed project focuses on the FEC concept.

2.3 Hamming Codes

Hamming codes are classical error-correcting codes that introduce parity bits into a block of data.

The Hamming (7,4) code takes four information bits and converts them into seven coded bits.

Therefore:

$$k = 4$$

and

$$n = 7$$

The code rate is:

$$R = \frac{k}{n}$$

$$R = \frac{4}{7} = 0.5714$$

The Hamming (7,4) code can correct a single-bit error within a codeword under standard decoding conditions.

The project uses Hamming (7,4) as the medium-strength error-correction technique. Hamming codes are a well-established family of error-correcting codes capable of correcting one error in a binary block.

2.4 LDPC Codes

Low-Density Parity-Check (LDPC) codes are advanced error-correcting codes based on sparse parity-check matrices.

LDPC codes provide strong error-correction performance while supporting high data throughput. They are widely used in modern communication systems.

In 5G New Radio, LDPC coding is used for important data channels.

However, implementing a complete LDPC encoder and iterative decoder is significantly more complex than implementing Hamming coding for a small academic simulation. Therefore, LDPC is studied in the literature review but is not directly implemented in the present project.

2.5 Polar Codes

Polar codes are another advanced class of error-correcting codes based on the concept of channel polarization.

Polar coding has been adopted for specific control-channel applications in 5G New Radio. 3GPP's channel-coding specification includes the coding procedures used in NR systems.

Polar codes provide excellent performance for certain block lengths and applications. However, their implementation requires more complex encoding and decoding algorithms than the requirements of this project.

Therefore, the proposed project uses Hamming coding as a simple and understandable demonstration of adaptive error correction.

2.6 Fixed vs Adaptive Error Correction

A fixed error-correction system uses the same coding technique regardless of channel condition.

For example:

Channel Condition

↓

Hamming (7,4)

↓

Transmit

An adaptive system instead changes the coding level:

Good Channel → Low Protection

Moderate Channel → Medium Protection

Poor Channel → Strong Protection

The adaptive approach attempts to balance:

- Reliability
- Redundancy
- Throughput
- Computational complexity

Research on modern channel coding also emphasizes the importance of balancing reliability, throughput, latency, and efficiency for different communication requirements.

2.7 Research Gap

Advanced error-correction schemes such as LDPC and Polar codes provide strong performance, but their implementation can be computationally complex for a small educational project.

The objective of this project is not to create a new error-correcting code. Instead, the project focuses on the **adaptive selection of error-correction strength**.

The research/implementation gap addressed by the project is:

A simple and understandable simulation that demonstrates how coding strength can be changed according to channel quality and how this affects BER, throughput, and redundancy.

The project therefore uses established coding techniques as building blocks and focuses on the adaptive control mechanism.

CHAPTER 3 - SYSTEM DESIGN

3.1 System Overview

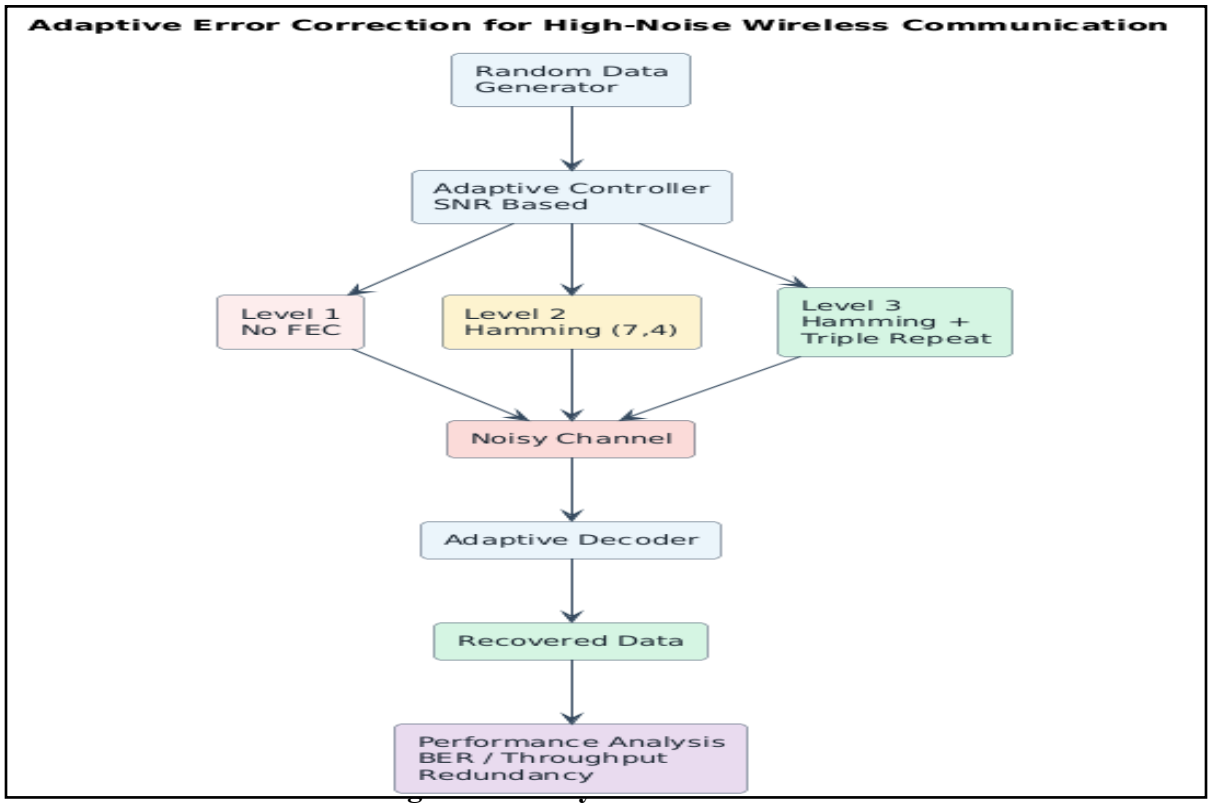
The proposed system is a software-based simulation of a wireless communication system.

The system consists of the following major stages:

1. Binary data generation.
2. SNR-based adaptive controller.
3. Error-correction encoding.
4. Noisy channel.
5. Error-correction decoding.
6. Performance measurement.
7. Result storage and analysis.

3.2 System Architecture

Figure 3.1: Proposed System Architecture



3.3 System Components

3.3.1 Data Generator

The data generator creates random binary information consisting of 0s and 1s.

The implementation uses:

DATA_SIZE = 10000 bits

This provides a sufficiently large sample for the simulation.

3.3.2 Adaptive Controller

The adaptive controller selects the coding level according to SNR.

The implemented decision rule is:

If SNR >= 8 dB

    Level 1

Else if SNR >= 4 dB

    Level 2

Else

    Level 3

3.3.3 Encoder

The encoder applies the selected error-correction technique.

Level 1:

Original Data → No Coding

Level 2:

Original Data → Hamming (7,4)

Level 3:

Original Data

    ↓

Hamming (7,4)

↓

Triple Repetition

3.3.4 Noisy Channel

The noisy channel introduces random bit errors into the encoded data.

The simulation uses a simplified relationship between SNR and bit-flip probability:

$$P_e = 0.5 \times 10^{-SNR/10}$$

where:

- P_e= simulated bit-flip probability
- SNR = signal-to-noise ratio in dB

As SNR increases, the simulated probability of bit corruption decreases.

Note: This is a simplified probabilistic channel model used for the project simulation. It is not a physical RF/AWGN implementation.

3.3.5 Decoder

The decoder recovers the original information.

For Level 1:

Received Data → Direct Output

For Level 2:

Received Data → Hamming Decoder

For Level 3:

Received Data

↓

Majority Voting

↓

Hamming Decoder

3.4 Adaptive Coding Levels

Level 1 – No Error Correction

Level 1 is selected when:

$$SNR \geq 8\,dB$$

The channel is considered sufficiently good for the simulation, so additional coding redundancy is avoided.

Coding expansion:

$$\frac{4}{4} = 1.00$$

Level 2 – Hamming (7,4)

Level 2 is selected when:

$$4 \leq SNR < 8$$

Four data bits are encoded into seven bits.

Therefore:

$$Redundancy = \frac{7}{4} = 1.75$$

This level provides moderate error protection.

Level 3 – Hamming + Triple Repetition

Level 3 is selected when:

$$SNR < 4\,dB$$

First, Hamming (7,4) encoding is performed.

Each resulting coded bit is then repeated three times.

Therefore:

$$Redundancy = \frac{7}{4} \times 3$$
$$Redundancy = 5.25$$

This provides the strongest protection among the three implemented levels.

3.5 Hamming (7,4) Working

The Hamming code uses seven positions:

Position: 1 2 3 4 5 6 7

P1 P2 D1 P4 D2 D3 D4

The parity bits are calculated using XOR operations.

$$P1 = D1 \oplus D\,2 \oplus D\,4$$
$$P2 = D1 \oplus D\,3 \oplus D\,4$$
$$P4 = D2 \oplus D\,3 \oplus D\,4$$

At the receiver, syndrome bits are calculated to determine whether a single-bit error has occurred.

The error position is calculated as:

$$ErrorPosition = S1 + 2S2 + 4S4$$

If the position is between 1 and 7, the corresponding bit is corrected.

3.6 Triple Repetition

Triple repetition is used to strengthen Level 3.

For example:

Original bit:

0 → 000

1 → 111

At the receiver, three received bits are examined.

The decoder uses majority voting:

000 → 0

001 → 0

010 → 0

011 → 1

100 → 0

101 → 1

110 → 1

111 → 1

This allows the system to tolerate some bit errors in the repeated sequence.

3.7 Noisy Channel Model

The project does not use physical wireless hardware. Instead, a software-based channel model introduces random bit flips.

The noise probability is calculated from SNR.

For example:

- Lower SNR → higher noise probability.
- Higher SNR → lower noise probability.

The received data can therefore differ from the transmitted data.

The purpose of this model is to provide a controlled environment for evaluating the adaptive error-correction mechanism.

3.8 Adaptive Controller

The controller is the main component responsible for adaptation.

Table 3.8.1: Decision Table

SNR Range	Coding Level	Protection
SNR < 4 dB	Level 3	High
4 dB ≤ SNR < 8 dB	Level 2	Medium
SNR ≥ 8 dB	Level 1	Low

The controller therefore follows the principle:

Use stronger protection when the channel is poor and reduce redundancy when the channel is good.

3.9 System Flowchart

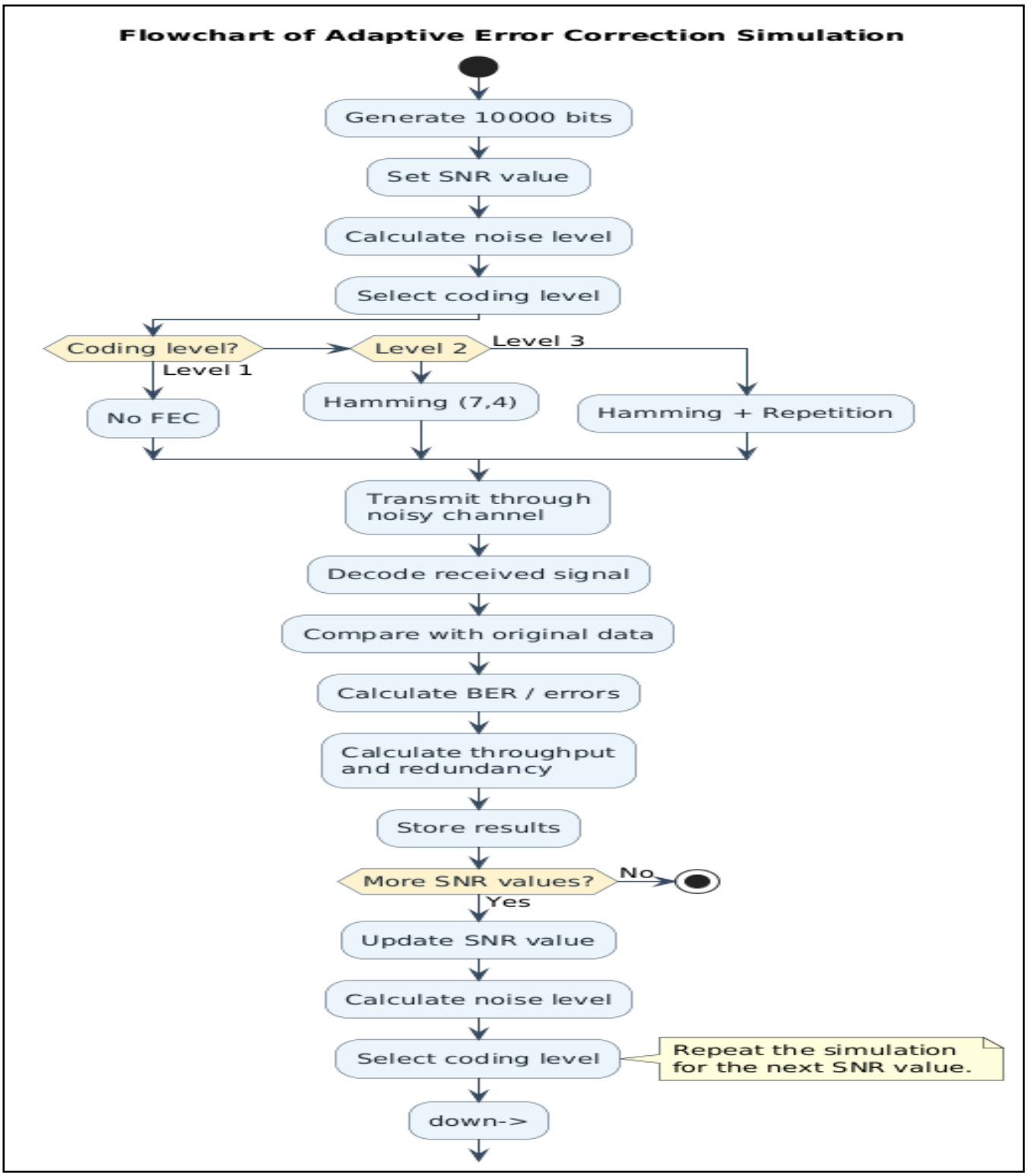


Fig 3.9.1: Flowchart

3.10 Algorithm

Algorithm: Adaptive Error Correction

Step 1: Start the simulation.

Step 2: Generate 10,000 random binary data bits.

Step 3: Select an SNR value.

Step 4: Calculate the corresponding channel noise probability.

Step 5: Select the coding level.

- If SNR ≥ 8 dB, select Level 1.
- If 4 dB ≤ SNR < 8 dB, select Level 2.
- If SNR < 4 dB, select Level 3.

Step 6: Encode the original data according to the selected level.

Step 7: Pass the encoded data through the simulated noisy channel.

Step 8: Decode the received data.

Step 9: Compare the decoded data with the original data.

Step 10: Calculate BER.

Step 11: Calculate throughput.

Step 12: Calculate redundancy.

Step 13: Store the result.

Step 14: Repeat the process for all SNR values.

Step 15: Repeat the experiment using fixed Hamming (7,4).

Step 16: Compare adaptive and fixed results.

Step 17: Stop.

3.11 Mathematical Formulation

Bit Error Rate

$$BER = \frac{Number\ of\ erroneous\ bits}{Total\ number\ of\ transmitted\ information\ bits}$$

In the implementation, BER is calculated after decoding, so it represents the **end-to-end post-decoding BER**.

Code Rate

$$R = \frac{k}{n}$$

For Hamming (7,4):

$$R = \frac{4}{7}$$

Redundancy Factor

$$Redundancy = \frac{Encoded\ bits}{Original\ information\ bits}$$

Therefore:

Level 1:

$$R_1 = 1.00$$

Level 2:

$$R_2 = \frac{7}{4} = 1.75$$

Level 3:

$$R_3 = \frac{7}{4} \times 3 = 5.25$$

Throughput

The effective throughput used in this simulation is:

$$Throughput = \frac{Successfully\ recovered\ information\ bits}{Transmitted\ coded\ bits} \times 100$$

This metric reflects both coding overhead and the number of successfully recovered bits.

CHAPTER 4 - IMPLEMENTATION

4.1 Software Requirements

The following software is required:

Table 4.1.1: Software Requirement

Software	Requirement
Operating System	Windows/Linux/macOS
Programming Language	Java
JDK	Java Development Kit
Compiler	javac
Runtime	Java Virtual Machine
IDE/Editor	VS Code / IntelliJ IDEA / Eclipse / Command Prompt
Output Format	Console + CSV

4.2 Hardware Requirements

The project is simulation-based and does not require dedicated communication hardware.

Minimum requirements:

- Computer/Laptop
- Processor: Dual Core or higher
- RAM: 4 GB or higher
- Storage: 100 MB free space
- Keyboard and display

Optional hardware is not required because the wireless channel is simulated using software.

4.3 Programming Language

The project is implemented using **Java**.

Java was selected because:

- It is platform independent.
- It provides object-oriented programming support.
- It provides built-in random number generation.
- It supports file handling for CSV output.
- It is suitable for implementing simulation algorithms.

- The program can be compiled and executed using standard JDK tools.

4.4 Program Modules

The final Java program contains the following major modules.

1. Data Generation Module

Method:

generateData()

Purpose:

Generates random binary information.

2. Hamming Encoder

Method:

hammingEncode()

Purpose:

Converts four information bits into seven coded bits.

3. Hamming Decoder

Method:

hammingDecode()

Purpose:

Calculates the syndrome and corrects a single-bit error.

4. Triple Repetition Encoder

Method:

tripleRepeatEncode()

Purpose:

Repeats each coded bit three times.

5. Triple Repetition Decoder

Method:

tripleRepeatDecode()

Purpose:

Uses majority voting to recover the original bit.

6. Adaptive Controller

Method:

selectAdaptiveLevel()

Purpose:

Selects the appropriate coding level based on SNR.

7. Channel Module

Method:

noisyChannel()

Purpose:

Simulates random bit corruption.

8. Performance Measurement

Methods:

calculateBER()

calculateThroughput()

calculateRedundancy()

countErrors()

Purpose:

Measures communication performance.

9. CSV Output Module

Methods:

writeHeader()

writeResult()

Purpose:

Stores simulation results for further analysis and graph generation.

4.5 Implementation Procedure

The implementation was carried out in the following sequence:

1. Create the Java source file.
2. Implement random binary data generation.
3. Implement Hamming (7,4) encoding.
4. Implement Hamming decoding.
5. Implement triple repetition encoding.
6. Implement majority-voting decoding.
7. Implement the noisy channel.
8. Implement SNR-based adaptive selection.
9. Implement BER calculation.
10. Implement throughput calculation.
11. Implement redundancy calculation.
12. Implement fixed Hamming comparison.
13. Implement CSV result generation.
14. Compile the Java program.
15. Execute the simulation.
16. Record the generated results.

4.6 Experimental Setup

The following SNR values were used:

$$0, 2, 4, 6, 8, 10, 12\ dB$$

For every SNR value, the system generates:

$$10{,}000$$

information bits.

Two experiments are performed:

Experiment 1: Adaptive Error Correction

The coding level changes according to SNR.

Experiment 2: Fixed Error Correction

Hamming (7,4) is used at every SNR value.

The two experiments are then compared using BER, throughput, redundancy, and errors.

4.7 Performance Metrics

BER

Measures the percentage/fraction of information bits that remain incorrect after decoding.

Lower BER indicates better error recovery.

Throughput

Measures effective successfully recovered information relative to the number of coded bits transmitted.

Higher throughput indicates better transmission efficiency.

Redundancy

Measures the expansion of data caused by error correction.

Lower redundancy indicates better bandwidth efficiency.

Errors

Represents the number of information bits that remain incorrect after decoding.

CHAPTER 5 - RESULTS AND ANALYSIS

5.1 Simulation Output

The final Java implementation was successfully compiled and executed using:

javac AdaptiveErrorCorrection.java

java AdaptiveErrorCorrection

The program generated two CSV files:

adaptive_results.csv

fixed_results.csv

The console output was also recorded.

5.2 Adaptive Error Correction Results

Table 5.2.1: Adaptive Simulation Results

SNR (dB)	Level	BER	Throughput (%)	Redundancy	Errors
0	3	0.504500	9.44	5.25	5045
2	3	0.247400	14.34	5.25	2474
4	2	0.206400	45.35	1.75	2064
6	2	0.099100	51.48	1.75	991
8	1	0.077300	92.27	1.00	773
10	1	0.047200	95.28	1.00	472
12	1	0.032600	96.74	1.00	326

5.3 Fixed Error Correction Results

For comparison, Hamming (7,4) was used at every SNR value.

Table 5.3.1: Fixed Hamming Results

SNR (dB)	Level	BER	Throughput (%)	Redundancy	Errors
0	2	0.500600	28.54	1.75	5006
2	2	0.346400	37.35	1.75	3464
4	2	0.195700	45.96	1.75	1957
6	2	0.095500	51.69	1.75	955
8	2	0.048100	54.39	1.75	481
10	2	0.021000	55.94	1.75	210
12	2	0.007300	56.73	1.75	73

5.4 BER Analysis

The adaptive system shows a general decrease in BER as SNR increases.

At 0 dB, the adaptive system produces a BER of:

$$0.5045$$

At 12 dB, the BER decreases to:

$$0.0326$$

This demonstrates the expected relationship between channel quality and communication errors: higher SNR results in fewer simulated bit errors.

The adaptive system does not necessarily produce the lowest BER at every SNR because its primary objective is to balance reliability with transmission efficiency.

For example, at 12 dB:

- Adaptive BER = 0.0326
- Fixed Hamming BER = 0.0073

The fixed Hamming system provides stronger protection at this point because it continues using redundancy even though the adaptive system has switched to Level 1.

This result illustrates the fundamental trade-off between reliability and efficiency.

5.5 Throughput Analysis

The adaptive system shows a significant increase in throughput as the SNR improves.

At 0 dB:

$$Throughput = 9.44\%$$

At 12 dB:

$$Throughput = 96.74\%$$

The major improvement occurs when the adaptive controller changes from stronger coding to lower coding levels.

At 8 dB, the system switches from Level 2 to Level 1:

$$Redundancy: 1.75 \rightarrow 1.00$$

As a result, throughput increases to:

$$92.27\%$$

At 12 dB, throughput reaches:

96.74%

This demonstrates that adaptive coding can significantly reduce coding overhead in relatively good channel conditions.

5.6 Redundancy Analysis

The adaptive system uses three redundancy levels:

Table 5.6.1: Redundancy Level

Coding Level	Redundancy
Level 1	1.00
Level 2	1.75
Level 3	5.25

At low SNR, Level 3 is selected because the simulated channel is highly noisy.

At moderate SNR, Level 2 is selected.

At high SNR, Level 1 is selected because additional redundancy is avoided.

Therefore, the project demonstrates dynamic adjustment of coding overhead.

5.7 Fixed vs Adaptive Comparison

The major advantage of the proposed system is not that it produces lower BER under every condition. Instead, it dynamically changes its coding overhead according to channel quality.

At 12 dB:

Adaptive system

BER = 0.0326

Throughput = 96.74%

Redundancy = 1.00

Fixed Hamming system

BER = 0.0073

Throughput = 56.73%

Redundancy = 1.75

The fixed Hamming system provides lower BER because it continues to apply error correction. However, it also transmits more coded bits.

The adaptive system sacrifices some error protection in good channel conditions to obtain substantially higher transmission efficiency.

This demonstrates the project's main concept:

The appropriate error-correction strength depends on the channel condition and communication objective.

5.8 Result Discussion

The simulation results confirm that the adaptive controller successfully changes the coding level according to SNR.

At:

0 dB → Level 3

2 dB → Level 3

4 dB → Level 2

6 dB → Level 2

8 dB → Level 1

10 dB → Level 1

12 dB → Level 1

This follows the predefined adaptive decision rules.

The results show three major observations.

Observation 1: BER decreases with increasing SNR

Higher SNR produces lower simulated channel noise and consequently fewer residual errors.

Observation 2: Throughput improves at high SNR

When the channel becomes better, the system reduces error-correction overhead. This produces higher throughput.

Observation 3: Adaptive coding changes redundancy

The system uses high redundancy when the channel is poor and low redundancy when the channel is good.

Therefore, the simulation successfully demonstrates the intended adaptive error-control concept.

5.9 Graphs to Include

The CSV files generated by the program can be used to create the following graphs.

Figure 5.1 – BER vs SNR

- X-axis: SNR (dB)
- Y-axis: BER
- Plot adaptive and fixed results.

Expected observation:

BER generally decreases as SNR increases.

Figure 5.2 – Throughput vs SNR

- X-axis: SNR (dB)
- Y-axis: Throughput (%)
- Plot adaptive and fixed results.

Expected observation:

Adaptive throughput increases significantly at high SNR because the system reduces coding redundancy.

Figure 5.3 – Redundancy vs SNR

- X-axis: SNR (dB)
- Y-axis: Redundancy factor

Expected adaptive pattern:

5.25 → 5.25 → 1.75 → 1.75 → 1.00 → 1.00 → 1.00

This graph clearly demonstrates the adaptive behaviour of the proposed system.

Figure 5.4 – Number of Errors vs SNR

- X-axis: SNR (dB)
- Y-axis: Number of errors

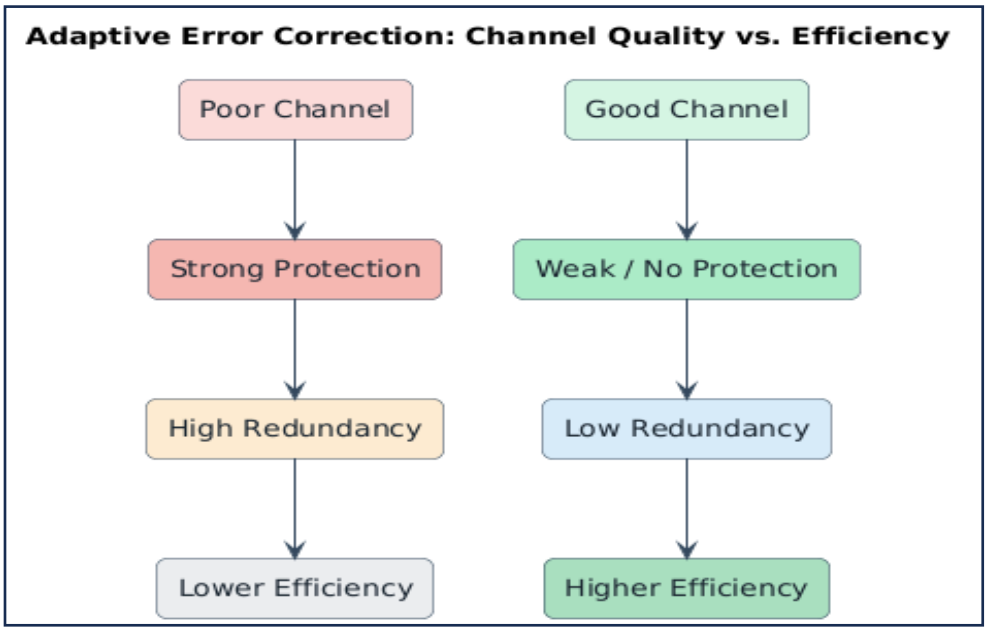
Expected observation:

The number of residual errors decreases as SNR increases.

5.10 Overall Result

The simulation successfully demonstrates an adaptive error-correction mechanism for varying wireless channel conditions.

The main result is the dynamic trade-off between protection and efficiency:



The system therefore avoids using the same coding strength for every channel condition.

CHAPTER 6 - CONCLUSION

6.1 Conclusion

The project **"Adaptive Error Correction for High-Noise Wireless Communication"** was successfully designed and implemented as a Java-based simulation.

The system dynamically selects the error-correction level according to SNR. Three coding levels were implemented: no error correction, Hamming (7,4), and Hamming (7,4) combined with triple repetition.

The simulation was performed using SNR values from 0 dB to 12 dB. Performance was evaluated using BER, throughput, redundancy, and number of residual errors.

The experimental results show that the adaptive controller correctly changes the coding strength according to the channel condition. At low SNR, stronger coding is selected, while at high SNR, coding overhead is reduced.

The most important result is that the adaptive system provides a mechanism for balancing reliability and transmission efficiency. For example, at 12 dB the adaptive system uses a redundancy factor of 1.00 and achieves 96.74% throughput, whereas the fixed Hamming system maintains a redundancy factor of 1.75 and achieves 56.73% throughput.

Although the fixed Hamming system provides lower BER at some high-SNR points, it continuously introduces coding overhead. The adaptive system instead changes the protection level based on channel conditions.

Therefore, the project successfully demonstrates the basic concept of **adaptive error-control coding for variable wireless channel conditions**.

6.2 Limitations

The current implementation has the following limitations:

1. The project is software-based and does not use physical wireless communication hardware.
2. The noisy channel is represented using a simplified probabilistic bit-flip model.
3. The system does not implement a complete physical-layer AWGN/BPSK communication chain.
4. The adaptive controller uses predefined SNR thresholds.
5. Advanced codes such as LDPC and Polar codes are not implemented.
6. Real-time channel estimation is not implemented.
7. Processing delay and energy consumption are not measured on physical hardware.
8. The simulation does not model practical effects such as multipath fading, Doppler shift, interference, or packet synchronization.

6.3 Future Scope

The project can be extended in several ways.

1. Real-Time Channel Estimation

Instead of directly providing SNR to the controller, the receiver can estimate channel quality using pilot or training symbols.

2. AWGN and Fading Channel

The current simplified channel can be replaced with:

- AWGN channel
- Rayleigh fading
- Rician fading
- Interference models

3. LDPC Implementation

The adaptive system can be extended to dynamically select LDPC coding rates for more realistic modern communication scenarios.

4. Polar Codes

Polar codes can be included as another coding option for short control messages.

5. Machine Learning

A machine-learning model could predict the best coding level from:

- SNR
- BER
- packet loss
- channel history
- retransmission rate

6. Real Wireless Hardware

The system can be integrated with wireless communication hardware such as:

- ESP32
- Software Defined Radio
- Wi-Fi modules

- LoRa modules

7. Real-Time Adaptive Communication

The controller can continuously monitor channel conditions and change coding strength during an active communication session.

8. More Performance Parameters

Future versions can analyze:

- latency
- energy consumption
- packet loss
- computational complexity
- spectral efficiency
- coding delay

6.4 Applications

The proposed concept can be applied to:

1. Wireless sensor networks.
2. IoT communication.
3. Mobile communication.
4. Wi-Fi communication.
5. Satellite communication.
6. Industrial wireless networks.
7. Emergency communication systems.
8. Vehicular communication.
9. Remote monitoring systems.
10. Low-power wireless devices.

REFERENCES

[1] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.

[2] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed., Pearson, 2004.

[3] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008.

[4] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed., Prentice Hall, 2001.

[5] R. G. Gallager, “Low-Density Parity-Check Codes,” *IRE Transactions on Information Theory*, vol. IT-8, no. 1, pp. 21–28, 1962.

[6] E. Arıkan, “Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels,” *IEEE Transactions on Information Theory*, vol. 55, no. 7, pp. 3051–3073, 2009.

[7] 3GPP, **TS 38.212: NR; Multiplexing and Channel Coding**, 3rd Generation Partnership Project. The specification covers NR channel coding and related processing.

[8] 3GPP, **5G NR Radio Interface – Channel Coding and Modulation**, 3GPP technical material describing the use of LDPC and Polar coding in 5G NR.

[9] IEEE, “Hamming Codes as Error-Reducing Codes,” *2016 IEEE Information Theory Workshop (ITW)*.

[10] IEEE, “Channel Coding Challenges for Short and Moderate Block Lengths-based Applications,” IEEE Conference Publication.

[11] IEEE, “Performances Comparison between Low Density Parity Check Codes and Polar Codes,” *2022 29th International Conference on Systems, Signals and Image Processing (IWSSIP)*.

ANNEXURE 1 - GITHUB UPLOAD SCREENSHOT

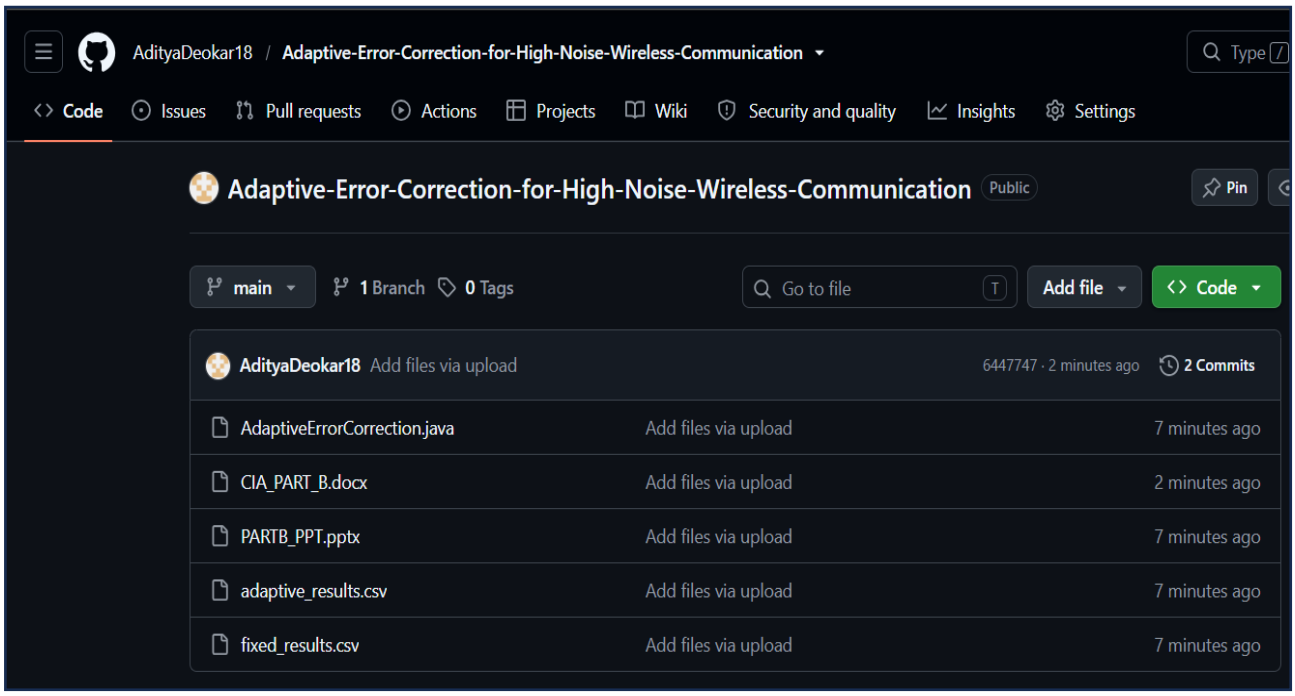
The complete Java source code and result files can be uploaded to a GitHub repository.

Suggested repository structure:

Adaptive-Error-Correction-for-High-Noise-Wireless-Communication/

```
|
├── AdaptiveErrorCorrection.java
├── adaptive_results.csv
├── fixed_results.csv
├── Presentation PPT
├── Report
```

Figure: GitHub repository containing the project source code and simulation results.



ANNEXURE 2 - PRESENTATION GEO-TAGGED PHOTO

A geo-tagged photograph of the project presentation/demo is attached as evidence of project presentation.

Figure: Geo-tagged photograph during project presentation.

